

Lab 01 — Debugging: Fix Four Planted Bugs

Module: COMP H4204 — Games Engineering II

1. Objectives

By the end of this lab you will be able to:

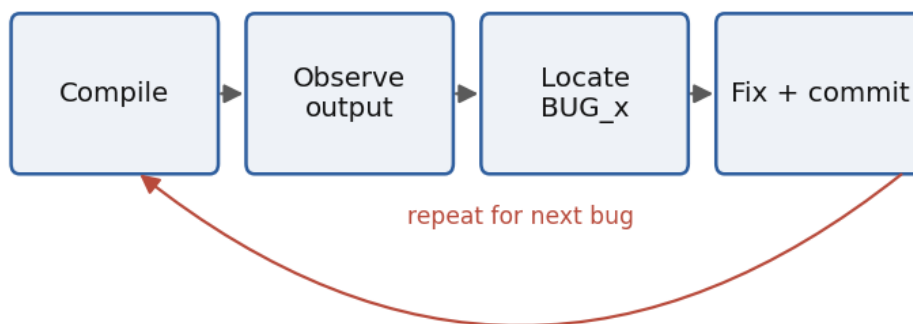
1. Compile and run a small C++ program with g++ from the VSCode terminal.
2. Read compiler diagnostics to locate a defect quickly.
3. Fix one defect at a time and commit each fix separately.

2. Learning outcomes

- **LO1** — Recognise four common beginner-C++ defects (operator swap, missing brace, integer division, deprecated header).
- **LO2** — Produce a GIT commit history a reviewer can read to reconstruct the debugging session.

3. Diagram

Lab 01 — debug cycle



Debug cycle

4. Problem statement

You are given lab01_debug.cpp. It defines two small arithmetic classes with add, subtract, multiply, and divide. When compiled, it either fails to build or produces incorrect output for the sample inputs. The source contains **four intentional defects**, each marked with a `//<BUG_x>` comment.

5. Tasks

1. Copy the starter into `StudentID/LabNN/lab01/`.
2. Compile: `g++ -std=c++17 -Wall -Wextra lab01_debug.cpp -o lab01_debug`. Read the compiler output.
3. Fix one bug at a time. Leave the `//<BUG_x>` marker in place and add a short note beside it, e.g. `//<BUG_1 fixed: swapped '-' for '+'>`.
4. After each fix, compile and run: `./lab01_debug`.
5. Commit each fix separately with a meaningful message.

Expected final output:

```
Addition    (10 + 5) = 15
Subtraction  (10 - 5) = 5
Multiplication (10 * 5) = 50
Division     (10 / 5) = 2
```

6. Lab Submission

Every lab in this module uses the same submission format. Place the following in `StudentID/LabNN/lab01/`:

1. **Updated source code** — `lab01_debug.cpp` with your fixes.
2. **Program output log** — `run_log.txt` (terminal output of a successful build and run).
3. **VSCode screenshot** — `screenshots/vscode.png` (your source open, file explorer visible).
4. **Terminal screenshot** — `screenshots/terminal.png` (build command followed by run output).
5. **Lab report** — `Lab01_Report.md`, `.docx`, or `.pdf`. 150–250 words on how you approached the lab, what you learned, and any difficulty.

7. Submission Deadline

The Lab work must be completed and pushed to your GIT by **Monday, i.e., 14 September 2026, 09:00 AM**